Safe to Drop: an Exact Criterion for Quantum Temporaries

ANONYMOUS AUTHOR(S)

We specify a safe quantum drop by observational equivalence to a program that prepares the dropped block independently of the rest, comparing all registers jointly just before the drop, with earlier drops deleted. With every unitary available as a gate, we show that drop safety is equivalent to restoration by one unitary on the dropped block alone. Without measurement, a drop of temporaries is safe exactly when one fixed pure state of the block, a single state vector, serves every input. With measurement, the criterion applies to each measurement record separately, for all inputs, keeping the weight with which the record occurs. We also consider loans, where the witness receives the unknown borrowed input instead of preparing a fixed state. Our restricted compiler computes numerical return gates and reuses wires, tested by independent simulation.

1 Specifying a safe drop

We want to reuse temporary registers when their lifetimes end, as we do classically by discarding their values. A unitary quantum subroutine cannot reset, but its gates are reversible, so we usually restore a temporary by uncomputation [1, p. 525][2], running the operations that produced it backwards [3, 14]. Silq and Unqomp automate this by letting only `qfree` gates, which create no superposition, produce the temporary [3, §4.4][14, §2.2], so both refuse to uncompute even two Hadamard gates, which each create superposition but together compose to the identity [14, §6.2]. We ask when one unitary on the dropped registers alone restores their allocation state and leaves the rest intact.

One `drop` may name several registers, which form its *block*, and one occurrence of `drop` is a *site*. Dropping removes names and applies no physical operation, so the dropped registers remain, only hidden. To compare two programs at a site by their joint state vectors we stop just before it and delete every earlier drop from the text, renaming reused names apart, so every register stays visible. This is the *exposed prefix*, and we write $p \simeq q$ when no closed context distinguishes those of p and q. A context supplies the input, possibly entangled with a reference of its own, may apply gates, and finally measures every register jointly with that reference, so it detects any correlation between the block and the rest.

Now take a program p that allocates a temporary t in a pure state ψ, works on it and the input v with measurement-free code M, and drops it. Beside p we put a witness $\mathcal{S}(R, U)$ that prepares t without consulting the input, by one gate R on t alone, and computes on v separately, by a gate U on v and fresh registers for any further temporaries of M:

$$p(v) = \texttt{new t @ } \psi \texttt{; M; drop t; return v},$$
$$\mathcal{S}(R,U)(v) = \texttt{new t @ } \psi \texttt{; R t; U v; drop t; return v}.$$

We call the drop safe if $p \simeq \mathcal{S}(R, U)$ for some fixed R and U, the same pair for every input x of v. Only M's action on $\psi \otimes x$ must factor, not its code. In the witness t ends in $R\psi$ whatever v held, so $R^\dagger$ on t returns it to ψ without touching v, and if $p \simeq \mathcal{S}(R, U)$ the same $R^\dagger$ restores p's temporary, because a context may itself apply $R^\dagger$ and look at t. Throughout, registers are finite-dimensional over $\mathbb{C}$, and programs and contexts may apply any unitary.

Consider two programs on a qubit v:

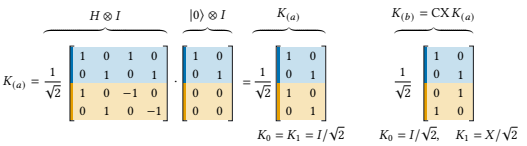
```
prog a (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;
  drop t; return v }
```

```
prog b (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t; CX t v;
  drop t; return v }
```

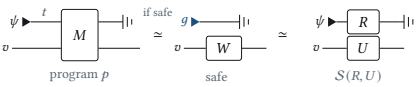
In program (a), H puts t in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ whatever v held, and another H restores $|0\rangle$ without touching v. Program (b) adds `CX t v`, flipping v when t is 1, so at input $|0\rangle_v$ it produces the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$, which independent operations cannot prepare. Both lexical lifetimes end at the drop, so checking lifetimes alone accepts both.

2 Checking the equivalence

Let K be the exposed prefix's map, taking input x to the joint state before the drop, block first. Program (a) gives $Kx = |+\rangle \otimes x = (|0\rangle \otimes x + |1\rangle \otimes x)/\sqrt{2}$, and program (b) gives $(|0\rangle \otimes x + |1\rangle \otimes Xx)/\sqrt{2}$, both pictured as matrices read from right to left. Reading t in its basis $|0\rangle, |1\rangle$, the part of K tagged by $|i\rangle$ is a matrix of its own, the *slice* K_i, blue where t reads 0 and gold where it reads 1, which is what the program has done to v in the branch where t reads i, so that $Kx = \sum_i |i\rangle \otimes K_i x$: unlike a classical `switch`, all branches are kept coherently.



The drop applies no gate and only removes access to the tags, so we are asking whether the temporary that carries them is left in one fixed pure state, independent of the input and uncorrelated with the rest. That needs every branch to do the same thing to the remaining registers, here v, up to a scalar, possibly zero, so the slices are proportional, $K_i = g_i W$, where the g_i are the coordinates of one fixed pure state g of the temporary and W is what the prefix did to the rest. In program (a) both slices are $I/\sqrt{2}$, so $g = |+\rangle$ and $W = I$, while in program (b) they are $I/\sqrt{2}$ and $X/\sqrt{2}$, which are not proportional. As pictured, a safe program, its factorization and its witness agree until the drop.



Theorem 2.1 (without measurement). *For a well-typed measurement-free program p at a drop of only allocated temporaries, let K be its exposed prefix's joint map and ψ the pure allocation state. The following are equivalent.*

(1) $p \simeq \mathcal{S}(R, U)$ for some independent witness.

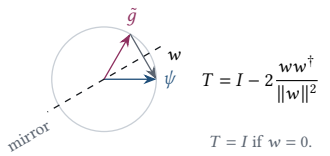
(2) There are a unit vector g on the block and a linear map W to all other registers such that $\forall x,\ Kx = g \otimes Wx$.

(3) One unitary on the block alone restores ψ at every input.

The map W is an isometry, preserving norms and inner products. The witness prepares $g = R\psi$ and realizes W through U acting on the input and fresh registers.

Deutsch–Jozsa shows why slices need only be proportional, not equal. With $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, its oracle obeys $O_f(|-\rangle \otimes |z\rangle) = |-\rangle \otimes (-1)^{f(z)}|z\rangle$, so the temporary stays fixed and the data carries the function's phase, and the two slices differ only in sign.

We call g a *certificate*. Since joint observations fix K only up to an overall phase, choose the representative $\tilde{g}$ with $\langle\psi, \tilde{g}\rangle$ real and nonnegative, moving every phase into W. If the overlap is zero, make the first nonzero coordinate of $\tilde{g}$ real and positive. If $\tilde{g} = \psi$, as safe use [12, Eq. (1)] and clean safety [11, Def. 1] require of every ancilla, no gate is needed. Otherwise the pictured Householder reflection [9, p. 340, Eqs. (3)–(4)] $T = I - 2ww^\dagger/\|w\|^2$ through the hyperplane orthogonal to $w = \psi - \tilde{g}$ exchanges $\tilde{g}$ and ψ. Applied to the temporary alone, T returns it to ψ at every input and leaves the rest unchanged: $\|w\|^2 = -2\langle w, \tilde{g}\rangle$ gives $T\tilde{g} = \tilde{g} + w = \psi$. Being its own inverse, it is also the witness's R. For program (a) it is H, and Grover's diffusion [8, §4] is $-T$ with w the uniform state.



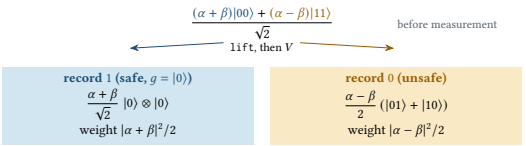
3 One measurement record at a time

With measurement, a program can be safe on one sequence of outcomes and unsafe on another. Our `lift`, after dynamic lifting [7] without its separate circuit-generation stage, measures a register and binds an outcome that only classical `if` guards can read. In `lift[{0}] t`, outcome 1 keeps the coordinates where t reads 0 and outcome 0 keeps their complement. A *record* o lists the outcomes along one path through the program, and its exposed map K_o is the joint map along that path, left unnormalized on purpose, so that at a normalized input x the probability of reaching the drop along that path is $\|K_o x\|^2$.

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>;
  H v; CX v t;
  b = lift[{0}] t;
  V t v; drop t; return v }
```

```
prog witness (v : [2]) : [2] {
  new t : [2] @ |0>; new a : [2] @ |0>;
  H v; CX v a;
  k = lift[{0}] a; drop a;
  drop t; return v }
```

Take the listed program (c) at input $x = \alpha|0\rangle + \beta|1\rangle$, with t first. Before the measurement it holds $((\alpha+\beta)|00\rangle + (\alpha-\beta)|11\rangle)/\sqrt{2}$, and V fixes $|00\rangle$ and sends $|11\rangle$ to $(|01\rangle + |10\rangle)/\sqrt{2}$, so the two records leave the unnormalized states $K_1 x$ and $K_0 x$ pictured with their weights.



On record 1 the slices are again proportional, with $W_1 x = (\alpha+\beta)|0\rangle/\sqrt{2}$, while record 0 holds a Bell state whenever $\alpha \neq \beta$ and is unsafe. Normalizing $W_1 x$ would hide record 1's weight, 1 at input $|+\rangle$ and only $1/2$ at input $|0\rangle$. To preserve these weights the measurement-free witness no longer suffices, because allocations and gates preserve length, so its map is isometric, while W_1, a *contraction* lengthening no vector, shrinks some. The witness, with $R = I$, puts the lost length on a fresh qubit a, preparing with U the same state on (a, v), and `lift[{0}] a` keeps outcome $k = 1$, where a reads 0, leaving exactly $W_1 x$ beside $|0\rangle_t$ with the same weight. Any contraction W is realized this way, as $x \mapsto |0\rangle \otimes Wx + |1\rangle \otimes (I - W^\dagger W)^{1/2} x$ preserves length and so extends to a gate. By $p \simeq_o \mathcal{S}(R, U)$ we mean equal contextual

observations on p's record o and on the witness's own kept outcome 1, omitting its fixed $|0\rangle_a$ factor, with neither run normalized, whatever happens on other records.

Theorem 3.1 (on a selected record). *Let the site s drop only allocated temporaries in a well-typed program p, and let o be a record whose path executes s. With every unitary available as a gate, let K_o be the exposed prefix's map at s on o, with the block first. The following are equivalent.*

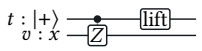
(1) $p \simeq_o \mathcal{S}(R, U)$ for some one-lift independent witness.

(2) There are a unit vector g on the block and a linear contraction W_o to all other registers such that $\forall x,\ K_o x = g \otimes W_o x$.

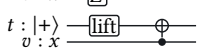
(3) One unitary on the block alone restores its pure allocation state ψ at every input on o.

If $K_o = 0$, all three conditions hold. Different records may need different return gates.

A caller who ignores the record sees only the *summed channel* [17, §2.2.1], which averages the records' states of the returned registers and a reference by their probabilities. It predicts every joint measurement of both, so one might expect it to decide safety, but it cannot do so on each record. For example, run the programs drawn below at input $x = \alpha|0\rangle + \beta|1\rangle$. Before `drop t` their unnormalized states on records 1 and 0 are



$|0\rangle \otimes x/\sqrt{2}$ $\quad$ $|1\rangle \otimes Zx/\sqrt{2}$ $\quad$ safe on both



$(\alpha|00\rangle + \beta|11\rangle)/\sqrt{2}$ $\quad$ $(\alpha|10\rangle + \beta|01\rangle)/\sqrt{2}$ $\quad$ safe on neither

Only the first row gives t one input-independent state per record. Yet both act on v as the same summed channel, leaving v in $|0\rangle$ or $|1\rangle$ with probabilities $|\alpha|^2$ and $|\beta|^2$, and every record has probability $1/2$ at every input, so only the per-record states tell them apart.

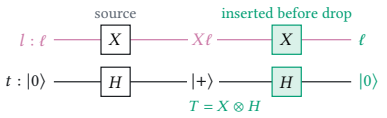
4 Blocks and loans

Dropping a tuple tests the named registers together. A Bell pair prepared in two fresh temporaries and left untouched is safe as a block of two, since the pair has one fixed state vector, though each half alone has the mixed state $I/2$ and so supplies no pure certificate.

We also consider blocks containing *loans*, registers borrowed in unknown states, possibly entangled with their caller, who expects them back as they were. The witness cannot prepare such a block, since it does not know the loan's state ℓ. Instead $\mathcal{S}(R, U)$ receives ℓ, allocates the temporaries beside it, the map $J\ell = \ell \otimes \psi$ with loans first, applies R to the whole block, and lets U act only outside it. The fixed certificate g, a map from $\mathbb{C}$ into the block, becomes an isometry $B : \mathcal{H}_{\text{in}} \to \mathcal{H}$ from the loans' space into the block's space $\mathcal{H}$ that carries the loan's input along. The drop is then safe, $p \simeq \mathcal{S}(R, U)$, exactly when $K = B \otimes W$. The witness exists because a unitary R of the block carries the isometry J onto B, $RJ = B$, and the return gate $T = R^\dagger$ satisfies $TB = J$. For program `mixed`, listed beside its circuit, with loan l, we have $B\ell = X\ell \otimes |+\rangle$, and the inserted $T = X \otimes H$ hands the unknown loan back to the caller at the drop and restores t, although `mixed` returns `()`.

With measurement we again compare one record at a time, using $B \otimes W_o$, where B and the return gate may depend on the record. If the caller's loan and reference together are correlated with the program's other inputs, the record can change their normalized

```
prog mixed (loan l : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X l;
  drop (l,t); return () }
```



joint state, which the return gate does not undo. If they are not, and always without measurement, the return restores that state on every record that occurs.

5 Scope and further work

The language is linear and first order. We prove progress and preservation for its abstract machine with lifting and classical guards, and extend it to restore a safe block on each record and pool its temporaries for a later `new` of matching dimension and state. Reuse preserves the records, their weights and, on each record, the reduced state of the returned registers. Our Haskell checkers compute exactly: one builds K_o at one-temporary sites, the other runs the program on basis inputs and their sum. Both grow exponentially in the number of qubits, but even with only basis allocations and X, `CX` and Toffoli gates, deciding whether a drop of temporaries is safe on a given record is coNP-complete in the size of the program text, as is deciding safety of all such drops on all records (compare [12, Cor. 3.1]).

Our OpenQASM 3 compiler uses the second check, computes numerical return gates, and reuses basis-allocated blocks of temporaries safe on every record, since records share one wire map, clearing them to $|0\rangle$. Replacing program (a)'s return with `new u : [2] @ |1>; return (v,u)`, typed `[2] * [2]`, then uses two circuit qubits where fresh allocation would use three: the return gate restores t before X prepares u on t's wire, and an independent simulation confirms that v keeps its Bell correlations with a reference and that u ends in $|1\rangle$. Allocations must be basis states, program (c)'s V is unsupported, and loans may be dropped only before lifts or classical guards. The general-case loan-return check tests $TB = J$ on rounded columns of B, using the computed matrix, not the emitted gates.

Testing what the temporary ends as, not how it was produced, our criterion accepts programs the syntactic checks of Section 1 refuse, though joint uncomputation can restore an entangled temporary only by changing what the caller sees. ReQWIRE verifies that an ancilla is restored [15], as Q# [13] and fractional types [4] require, and Qutes inverts a temporary's history once it is dead or output-isolable [6, Defs. 2–6]. Local rotations repair separable errors on individual dirty ancillas [11, §5.2, Alg. 2], where our one gate per record restores a whole block. Twist checks purity [19], but a temporary swapped with the input is pure at each product input yet supplies no fixed state. On measurement-free circuits with clean qubit ancillas, slices summing to a unitary [12, Def. 3.1] is incomparable with safety: those of program (a) sum to $\sqrt{2}I$, proportional to a unitary but not norm preserving, though its drop is safe, and those of `new t : [2] @ |0>; CX v t; drop t` sum to I though its drop is unsafe. QBorrow needs an identity loan factor per denoted operation [16, Def. 5.1], Bona one per borrowing body [18, Def. 3.2], where our inserted return gate undoes any isometric encoding B. Loops and quantum recursion, re-entering scopes under quantum guards, remain open with the certificates they need, as do exact or approximate returns in a finite gate set and the question of when restoration beats measurement and reset [5, 10].

References

[1] Charles H. Bennett. 1973. Logical Reversibility of Computation. *IBM Journal of Research and Development* 17, 6 (1973), 525–532. doi:10.1147/rd.176.0525

[2] Charles H. Bennett. 1989. Time/Space Trade-Offs for Reversible Computation. *SIAM J. Comput.* 18, 4 (1989), 766–776. doi:10.1137/0218053

[3] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 286–300. doi:10.1145/3385412.3386007

[4] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[5] Matthew DeCross, Eli Chertkov, Megan Kohagen, and Michael Foss-Feig. 2023. Qubit-Reuse Compilation with Mid-Circuit Measurement and Reset. *Physical Review X* 13, 4 (2023), 041057. doi:10.1103/PhysRevX.13.041057

[6] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [quant-ph] Numbered as in arXiv:2603.14538v1.

[7] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 11 (2023), 26 pages. doi:10.1145/3571204

[8] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[9] Alston S. Householder. 1958. Unitary Triangularization of a Nonsymmetric Matrix. *J. ACM* 5, 4 (Oct. 1958), 339–342. doi:10.1145/320941.320947

[10] Fei Hua, Yuwei Jin, Yanhao Chen, Suhas Vittal, Kevin Krsulich, Lev S. Bishop, John Lapeyre, Ali Javadi-Abhari, and Eddy Z. Zhang. 2023. CaQR: A Compiler-Assisted Approach for Qubit Reuse through Dynamic Circuit. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS 2023)*. 59–71. doi:10.1145/3582016.3582030

[11] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16 Numbered as in arXiv:2608.13099v1.

[12] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. *Proceedings of the ACM on Programming Languages* 10, OOPSLA2 (2026), 2270–2298. doi:10.1145/3839528 Numbered as in arXiv:2608.09578v2.

[13] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[14] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[15] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[16] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. Borrowing Dirty Qubits in Quantum Programs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. 274–289. doi:10.1145/3779212.3790134 Numbered as in arXiv:2508.17190v1.

[17] John Watrous. 2018. *The Theory of Quantum Information*. Cambridge University Press. doi:10.1017/9781316848142 Numbered as in the author's 2018 prepublication draft.

[18] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL] Numbered as in arXiv:2608.08765v1.

[19] Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: Sound Reasoning for Purity and Entanglement in Quantum Programs. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 30 (2022), 32 pages. doi:10.1145/3498691